02 起步

Part I Foundations

本章定位

以穩定插入與明確邊界合併建立迴圈／歸納證明，逐步核對洞、record保存、inversions與RAM成本，再對任意長度推導merge sort的時間及峰值空間。

前置知識

陣列、迴圈不變量、基本遞迴與對數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用迴圈不變量證明插入排序正確。
- 從 RAM 模型的逐行成本建立最壞情況界限。
- 用分治遞迴描述合併排序並求解 T(n)=2T(n/2)+Theta(n)。
- 2.1 Insertion sort
- 2.2 Analyzing algorithms
- 2.3 Designing algorithms

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 Insertion sort

本章排序對象是records，每筆具有可比較的key與不變identity；keys形成total order，單次比較、搬移一筆record均為常數成本。輸出須keys非下降，且每筆原record恰出現一次；stable另要求等key保留原先後。索引採1起算，空區間不讀取元素。

incremental方法維護已排序prefix，再將下一筆插入。in-place表示除輸入外只需常數個word；不表示陣列在每個中途時刻都已排序，也不表示暫時沒有複製值。

以下and由左向右short-circuit。key是暫存record的值；若record是物件參照，只搬移參照、不修改物件內容。n≤1直接返回，避免把空陣列套進「第一筆已排序」的初始化。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

穩定的原地插入

```
INSERTION-SORT(A)
    n = A.length
    if n <= 1: return
    j = 2
    while j <= n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
        j = j + 1
```

嚴格大於才右移：等key不跨過新record。i=0時不求值A[i]，所以不越界。內層每次i減1，至多j−1次；外層j每次加1，至多n−1次，兩層均終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

外層不變量：有序與保存要一起證

令S是呼叫開始時的輸入副本。每次外層判斷前：A[1..j−1]是S[1..j−1]的**穩定排序**，而A[j..n]=S[j..n]。此處identity區分重複key，避免只證「值看起來相同」。

初始化j=2，單筆prefix本就有序且穩定，suffix未變。維護：內層只搬移prefix中嚴格大於key的尾段，再把key放在其左邊；舊prefix內相對先後未變，等key全留在新record前。故prefix擴成S[1..j]的穩定排序，suffix仍未變。

終止時j=n+1，不變量給A[1..n]是全部輸入的穩定排序。n≤1由提前返回獨立成立。不變量證明結果，前頁的遞減／遞增界證明一定到達結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

內層真正保存的是「陣列扣掉洞，再加key」

固定一次j，令B為該次開始的A[1..j]，B[1..j−1]已排序。把h=i+1視為邏輯洞，洞內記憶體的舊值不計入資料表示。

每次內層判斷前：A[1..i]=B[1..i]；A[i+2..j]=B[i+1..j−1]，這段每個key都嚴格大於暫存key=B[j]。非洞位置連同暫存key，恰保存B的全部records。初始化i=j−1，右段為空。

若A[i]>key，將A[i]複製到洞，i減1，新洞左移一格；上述兩段等式仍成立。若i=0或A[i]≤key，左段均≤key、右段均>key，填洞便有序且保存全部records。直接聲稱「每次shift後整段A仍是permutation」是錯的：複製造成的重複須由洞的表示消除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一組records的完整外層結果

輸入[(7,a),(2,b),(5,c),(2,d),(1,e)]，以7a等簡記；identity不是排序次鍵。

j / 暫存key	由右至左的key比較	右移次數h_j	填洞後A
2 / 2b	7>2真；到邊界	1	2b,7a,5c,2d,1e
3 / 5c	7>5真；2>5假	1	2b,5c,7a,2d,1e
4 / 2d	7>2真；5>2真；2>2假	2	2b,2d,5c,7a,1e
5 / 1e	7>1、5>1、2d>1、2b>1皆真；到邊界	4	1e,2b,2d,5c,7a

2b始終先於2d。共8次右移、10次key比較；while整個guard共12次求值，因i=0的兩次不執行key比較。這三種計數不可混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次插入內的每個狀態

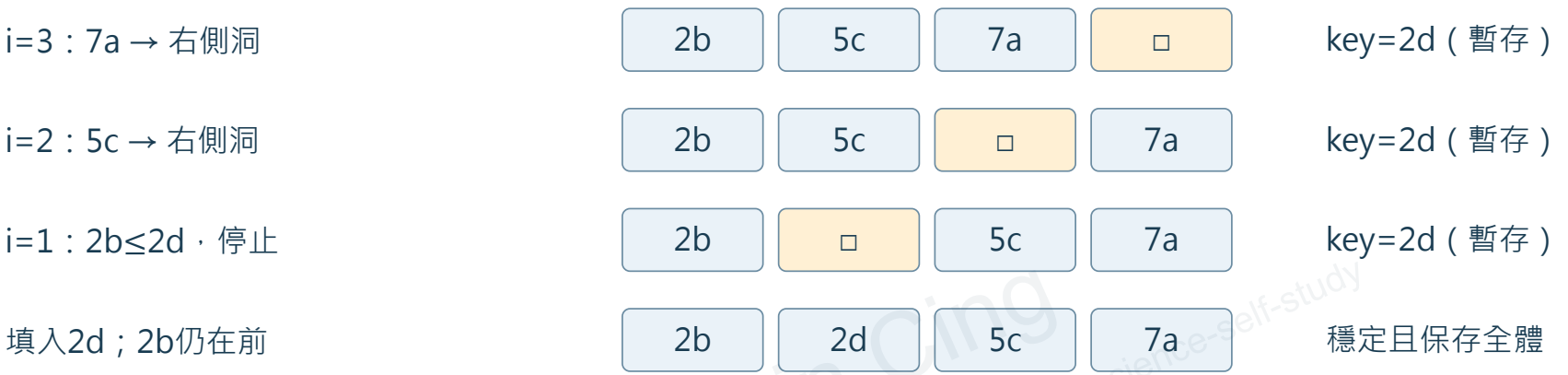
固定j=4，B=[2b,5c,7a,2d]，key=2d；□只表示邏輯洞，並非額外寫入陣列。

guard前i	非洞表示	動作
3	[2b,5c,7a,□] + key2d	7>2，A[4]←7a
2	[2b,5c,□,7a] + key2d	5>2，A[3]←5c
1	[2b,□,5c,7a] + key2d	2>2為假，停止右移
1	[2b,2d,5c,7a]	A[2]←key，填洞

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次插入內的每個狀態 （續）

同一次j=4：先保存key，再右移、填洞



□是被排除的邏輯位置；實體記憶體可仍含舊值，不能把重複值當成兩筆資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

偽碼的參數、別名與控制語義

本章函式傳參是值傳遞；陣列值是儲存體的reference，故A[i]賦值修改呼叫者看得到的陣列。B=A複製reference，兩者alias同一儲存體；B=copy(A)才有獨立陣列。將局部A重新綁到新陣列，不會把呼叫端的變數也改綁。

例如A=[3,1]，B=A後B[1]=8，A也成[8,1]；若改成B=copy(A)，B[1]=8不影響A。record identity與記憶體位址不是同一個概念；排序可重排references而不重建物件。

縮排界定block；return結束本次呼叫。NIL表示無物件，不是合法陣列位置。本章while的最後一次失敗guard也計成本；自然語言「排序某集合」若隱含非定長子程序，不能算一個常數步驟。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.2 Analyzing algorithms

單處理器word-RAM依序執行load/store、比較、分支與有限word算術。陣列位置可在常數時間存取；word至少能表達索引，典型$w=\Theta(\lg(n+1))$ bits，不能把任意長整數塞進一個word再宣稱任意運算$O(1)$。

這個模型沒有直接計cache miss、page fault、I/O及平行工作；實測時間還取決於實作與硬體。若key是長字串，比較可能要逐字讀取；本章常數比較前提就須另改。若中間算術溢出word，必須指定多word表示及其成本。

輸入大小依問題選：排序n筆records；L-bit整數以L衡量；圖通常需(V,E)兩個參數。數值N與編碼長度$\lg N$不可互換。以下成本是primitive operations，不是秒數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把每行執行次數寫出來

對$n \geq 2$，令h_j為第j次插入的右移數，$t_j = h_j + 1$為內層guard次數，$I = \Sigma h_j$。本章while版本的外層$j=2$另計一次，外層guard共n次。

操作	次數
n設定、提前返回判斷、j初始化	各1
外層guard	n
key暫存、i初始化、最後填洞、j加1	各$n-1$
內層guard	$I+n-1$
右移$A[i+1] \leftarrow A[i]$、i減1	各I

每操作成本$c_s > 0$、與n無關時，$T(A) = c_0 + c_1 n + c_2(n-1) + c_3(I+n-1) + c_4 I$，這裡$c_2$合併四項、$c_4$合併兩項。故$T(A) = \Theta(n+I)$；包含$n=0/1$可寫$\Theta(1+n+I)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為什麼右移數恰等於inversions

inversion是原輸入的索引對(u,v)，u<v且key(S[u])>key(S[v])，總數I(S)。相等key不形成inversion。

第j次插入前，prefix保存前j−1筆。它們中key大於S[j]者恰是排序prefix的一個尾段；演算法逐筆右移此尾段，各一次，其他record不移。因此h_j等於右端點v=j的inversions數。每個inversion有唯一右端點，Σh_j=I(S)，不是只對某個數例成立。

前例右端點j=2,3,4,5分別有1,1,2,4對，I=8。已排序輸入I=0，時間Θ(n)；嚴格反序且keys相異時I=n(n−1)/2，時間Θ(n^2)。只有少量inversions的輸入可接近線性，這是adaptive性質。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較次數、寫入次數與最佳／最差

對每j，key比較數為h_j+1（若h_j<j−1，最後比較為假），或h_j（若一路到i=0，由short-circuit結束）。設b為插入到最左端的次數，則C_key=I+n−1−b；整體guard=I+n−1；array writes=I+n−1，暫存key的寫入不算array write。

已排序時h_j=0、b=0，C_key=n−1；相異反序時h_j=j−1、b=n−1，C_key=n(n−1)/2。輸入長度相同也可有不同成本。

令$T_{\text{best}}(n) = \min_{|x|=n} T(x)$，$T_{\text{worst}}(n) = \max_{|x|=n} T(x)$。最佳／最差指定的是哪個函數；O/Ω/Θ指定對函數的漸近界。不能把O當「最差」或Ω當「最佳」的同義詞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平均成本必須先給分布

假設n個**相異**key的n!個排列等機率。每對原位置u<v，令X_uv=1當S[u]>S[v]，否則0；交換這兩位置是一一對應，所以Pr(X_uv=1)=1/2。

$I = \sum_{u<v} X_{uv}$，期望線性性給E[I]=ΣE[X_uv]=n(n−1)/4。這裡不需要各X獨立，事實上共用位置的指標通常相依。由T(A)=Θ(1+n+I)，且界中常數對每個輸入相同，E[T]=Θ(n^2)（n→∞）。

這不是「任何真實資料平均都平方」：若分布集中於已排序輸入，期望是Θ(n)。隨機輸入的期望與固定輸入、演算法自己抽亂數的期望，分母不同，必須分開陳述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從確切公式到成長階

對固定RAM costs，最差時間可整理成an^2+bn+c且$a>0$。取足夠大n，使$|b|n+|c| \le (a/2)n^2$，則$(a/2)n^2 \le T_{worst}(n) \le (3a/2)n^2$；這才支持$\Theta(n^2)$。

低階項和常數會影響有限n的快慢，Θ只說最終同階。$T_1=1000n$與$T_2=n^2$在$1<n<1000$時T_2較小，在$n>1000$時T_1較小；不等式前須使用同一成本單位。

空輸入與單筆輸入仍要正確處理；漸近忽略有限多n，不代表程式可以忽略它們。最佳常數時間的提前返回檢查，也不能自動把其他所有輸入的成本改善。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 Designing algorithms

divide-and-conquer把問題分為嚴格更小的同類問題，遞迴求解，再合併答案。只說「分兩半」不夠：須定邊界、base case、合併前提及結果。merge sort取A[p..r]，長m=r−p+1；p>r表示空區間。

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

合法非空呼叫要求1≤p≤r≤n；整陣列呼叫(1,n)，n=0時(1,0)直接返回。左右長分別ceil(m/2)、floor(m/2)，m≥2時都小於m。q寫成差值式避免p+r溢出的常見實作風險，但仍須確保合法索引範圍可表達。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

不依賴額外key值的MERGE

前提$1 \le p \le q < r \le n$，兩半已穩定排序。L、R是**獨立副本**，不能是會被覆寫A影響的alias。
$len(L)=q-p+1$，$len(R)=r-q$。

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key
            A[k] = L[i]; i = i+1
        else
            A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相等先取L，因原輸入左半records全先於右半，穩定性得以延續。只檢查長度不需key域之外的∞。

MERGE的保存、順序與終止證明

主迴圈前，k−p=(i−1)+(j−1)；A[p..k−1]恰是已消耗的L[1..i−1]、R[1..j−1]之穩定合併，已排序且其中每個key≤所有未消耗key。其餘未消耗records仍在獨立L/R中。

初始化空輸出成立。若兩邊非空，各自首筆就是各邊最小未消耗值；選較小者（等值選左）恰為全體下一筆。消耗對應一筆、k加1，保持長度等式、保存、順序與穩定性。未消耗總數每次減1，故終止。

退出主迴圈時至少一邊耗盡；另一邊已排序且不小於輸出尾，所以逐筆接上保留同一不變量。最後i=len(L)+1、j=len(R)+1、k=r+1，全部m筆恰輸出一次。若不複製L/R，A[k]可能覆寫尚未讀的輸入；證明所需的剩餘資料就消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有重複key的完整合併

L=[1a,4b,4c]、R=[2d,4e,6f]，原輸入左段先於右段。表列主迴圈每次比較；i,j為比較前的1-based位置。

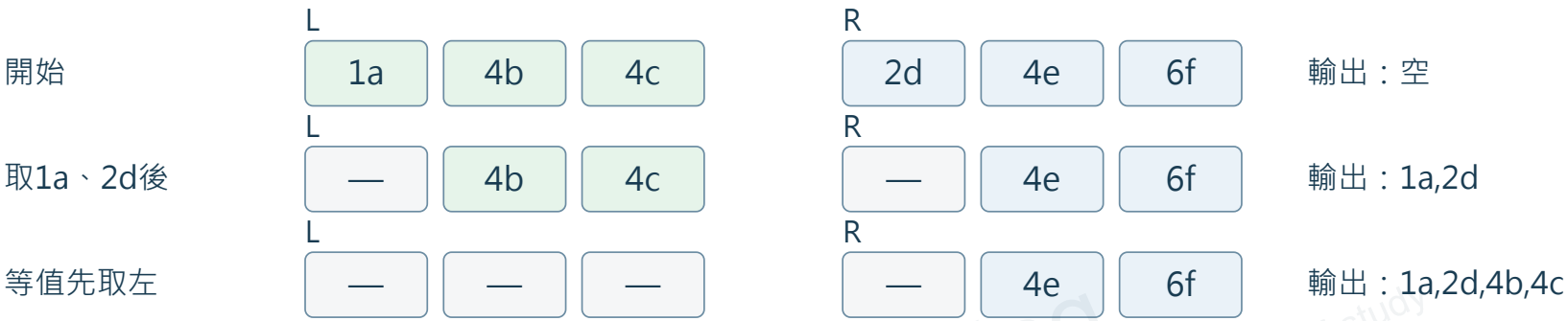
i,j	比較	取出後的輸出prefix
1,1	1a≤2d	1a
2,1	4b>2d	1a,2d
2,2	4b≤4e	1a,2d,4b
3,2	4c≤4e	1a,2d,4b,4c
4,2	L耗盡，不再比較key	接上4e,6f

共4次key比較、6次輸出寫入，另有6筆copy進L/R。若最後兩筆交錯耗盡，最多m−1=5次key比較；「MERGE是Θ(m)」不表示每次恰做m次key比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有重複key的完整合併 （續）

消耗一筆，輸出一筆；兩邊副本不會被A回寫破壞



L耗盡 → 依序接4e、6f。相同key的4b、4c、4e保持原本從左至右的先後。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sentinel是有前提的另一種表示

若使用sentinel，可在L/R尾各加一筆key=∞，固定m次從較小首筆取出，不用每次檢查耗盡。但∞必須嚴格大於每個合法key，且不是合法record。將最大合法整數當∞，遇到同值輸入就可能錯取sentinel，甚至下一步越界。

例如合法key允許+∞，L=[+∞]、R=[+∞,+∞]，加上相同∞哨兵後「等值選左」可能連取左真record、左sentinel，第三次便讀超過L。這不是正確的域外標記。

本章長度檢查版對任意total-order key域都可用，代價只是明確處理耗盡。若改為tagged sentinel，使它的排序順位嚴格高於所有real record，也須保證sentinel絕不輸出成資料。

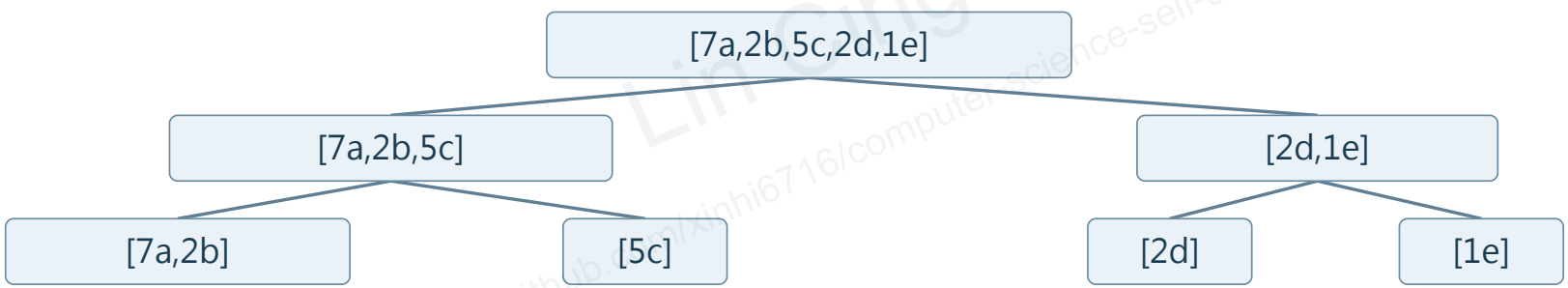
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從局部合併推到整個排序

對區間長m作strong induction。m≤1已排序且不變。m≥2時兩個子問題都嚴格較小，歸納假設使兩半各自穩定排序且保持各半全部records；MERGE前提成立。前頁證明使整段穩定排序，區間外未改，歸納完成。

終止由子問題長度遞減至0/1保證；正確性不依賴n是2的冪。輸入[7a,2b,5c,2d,1e]分為長3、2，再分2/1與1/1；各節點合併結果如下圖。左側2b與右側2d相等，根合併仍先2b。

五筆record：向下分割，向上回傳穩定排序結果



自底向上： [2b,7a] → [2b,5c,7a]；右半 → [1e,2d]

根合併： [1e,2b,2d,5c,7a]；相等key2：左2b先於右2d

左長3、右長2；單筆直接返回。圖省略[7a,2b]再分為兩個單筆的最後一層。

遞迴式的成本不能漏掉合併

對m≥2，分割計q為Θ(1)，兩個遞迴呼叫成本相加，MERGE複製m筆並輸出m筆，為Θ(m)，故

T(m)=T(ceil(m/2))+T(floor(m/2))+Θ(m)，T(0),T(1)=Θ(1)。

先對m=2^h採精確代理式U(1)=d、U(m)=2U(m/2)+cm（c,d>0）。深度ℓ=0..h−1有2^ℓ個internal nodes，每個成本cm/2^ℓ，該層總cm；h層合cmh。最底m個葉各d，故U(m)=cm lg m+dm。

真成本每internal node夾於c_low・size與c_high・size、葉成本也有固定上下界，將兩者各自代入便得到Θ(m lg m)，而非假設同一c恰代表所有指令和base case。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

任意長度的界與一個六筆成本表

平衡拆分到深度ℓ後，各未終止區間長度至多ceil(m/2^ℓ)，所以height≤ceil(lg m)。同一深度的區間互不重疊，總長至多m，給internal成本O(m lg m)。

在深度ℓ<floor(lg m)，每段至少floor(m/2^ℓ)≥2，尚無葉；各段總長恰m，給floor(lg m)層各Ω(m)。m≥2時floor(lg m)≥(lg m)/2，故下界Ω(m lg m)。這是任意m的雙邊證明。

m=6的深度	問題長度	internal長度總和
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1（另兩葉已在上層）	0

本例copy進buffers與回寫A各16筆，不是把每一層都誤算成6筆；base case共6個。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

峰值空間不是所有配置次數的總和

本版MERGE在兩次遞迴返回後才建立L/R，完成合併即可釋放；單次長m的buffers合m筆。子呼叫的buffers不與父MERGE buffers同時存活，因此峰值輔助空間Θ(n)，另有Θ(lg n)呼叫stack，合仍Θ(n)（n≥2）。

整趟累計buffer配置量可以是Θ(n lg n)，但它不是同時需要的記憶體。若實作保留每個已完成呼叫的buffer不釋放，就換了生命期模型；不能套用上述峰值結論。也可預先配置n筆scratch反覆重用，並保留同樣的非alias前提。

insertion sort只需key、i、j等常數額外word，較省空間；merge sort在本實作保證Θ(n lg n)時間但非in-place。兩者都穩定，前提分別是strict >右移與等值先取左。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 02。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study